경영하는 코딩 셰프

박명훈

개발자 박명훈

010.4183.1640

INDEX

01 자기소개 및 경력

02 기술

03 OPEN AI API와 LANGCHAIN 모델을 활용한 맞춤형 취업준비 서비스 — JOBIS

04 OPEN AI API를 활용한 AI 코딩 학습 도우미 — CEnter

05 마무리

박명훈

PARK MYUNG HOON
1998.06.12
010 4183 1640
audgns1640@gmail.com

01 학력사항

2023. 02	원광대학교 경영학부 졸업
2017. 02	조선대학교부속고등학교 졸업

02 교육사항

2024.11.12~2025.05.13	스마트인재개발원 DCX기반 빅데이터 분석서비스 개발자 심화 과정 수료 (960시간)

03 경력사항

2021.06~2024.09	요쿠야 & 카이치 텐동,라멘,테판야끼 전문점 실장

04 대외활동

2022	원광대학교 41대 경영대학 부학생회장
2021	원광대학교 40대 경영대학 국제교류국장
2017	원광대학교 21대 경영학부 사무국 팀원

05 자격사항

2025.03.12	정보처리기사 필기
2016.12.15	1종 보통

06 입상

2025.05.13	스마트인재개발원 실전융합 프로젝트 장려상
2025.05.13	스마트인재개발원 DCX기반 빅데이터 분석서비스 개발자 심화과정 모범상
2024.11.28	스마트인재개발원 JAVA 페스티벌 팀 2등
2023.02.20	원광대학교 경영대학장 공로상

"안녕하세요. 경영학을 전공하고 요식업의 고객 중심 경험을 쌓은 후,
데이터와 AI 기반 백엔드 개발자로 성장하고 있는 박명훈입니다."

- **경영학** — 경영학 전공, 경제·금융 원리와 실제 주식 투자 경험을 바탕으로 데이터 분석과 문제 해결의 사고력과 강점을 가지고 있습니다.
- **요식업** — 고객 중심의 서비스를 통한 문제 해결 능력을 키웠습니다.
- **프로그래밍** — 스마트인재개발원에서 AI 및 백엔드 개발을 학습하며, 데이터 기반 서비스 개발 역량을 키우고 있습니다.

GitHub : https://github.com/maeng-hun

Back-end

Java

FastAPI

Front-end

JS

DB

PostgreSQL

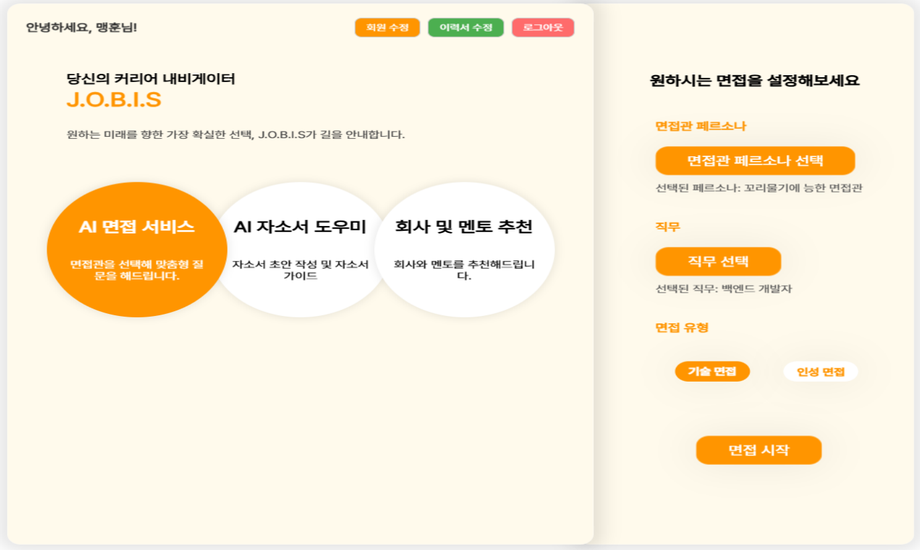
ORACLE

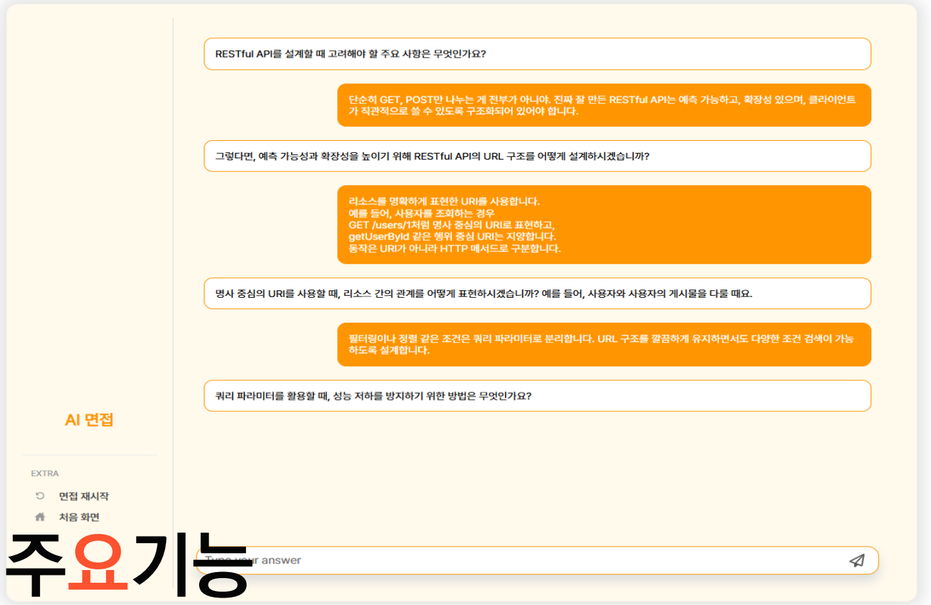
MySQL

Tech Stack

J.O.B.I.S

주제	LLM과 LANGCHAIN 모델을 활용한 맞춤형 취업준비 서비스
개발기간	2025.04.14 ~ 2025.05.12
개발인원	4명
개발환경	VS Code, LLM, LangChain, PostgreSQL
서버/API	FastAPI, REACT, OpenAI API, ChromaDB, Embedding
사용언어	Python, JavaScript, TypeScript, SCSS





주요기능

01. 키워드만 입력하면 AI가 자기소개서 초안을 자동으로 만들어줍니다.

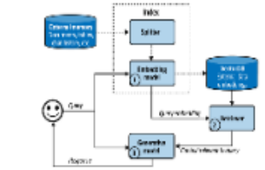
02. 작성한 자기소개서에 대한 피드백도 AI가 자동으로 제공합니다.

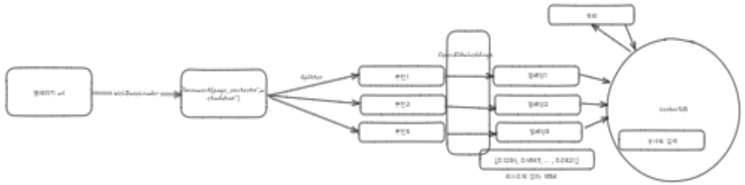
03. 자기소개서를 기반으로 면접관 유형과 질문 스타일을 고를 수 있습니다.

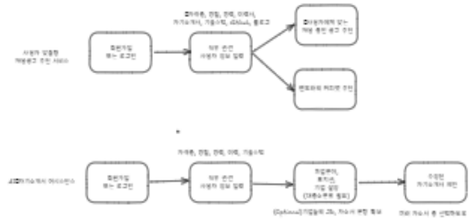
04. 사용자의 자기소개서·포트폴리오 데이터를 벡터DB에 저장하고, 유사도 분석을 통해 어울리는 회사와 멘토를 추천합니다.

GitHub https://github.com/2024-SMHRD-DCX-BigData-16/J.O.B.I.S.

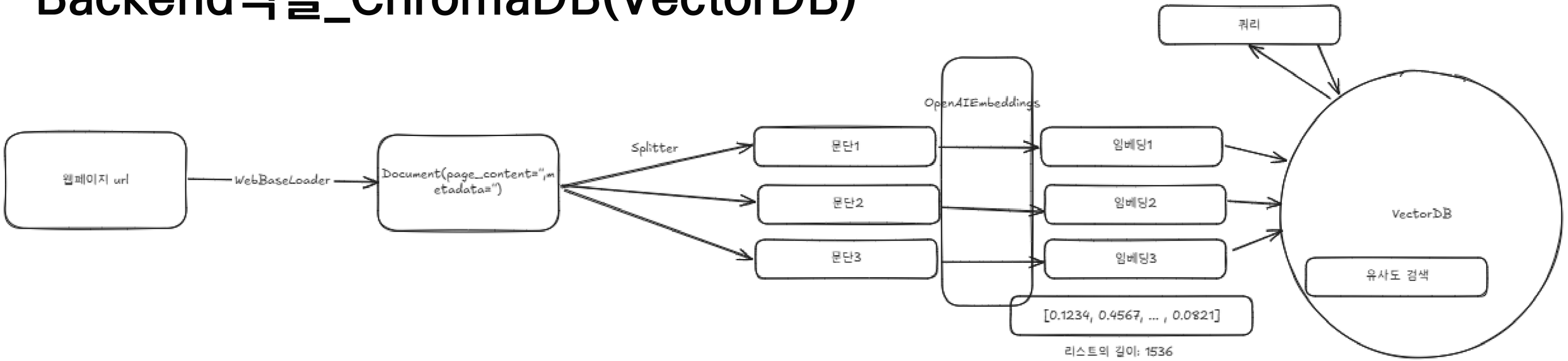
시스템 아키텍처 QR코드 봐주시면 감사하겠습니다! (Join Room 클릭!)







Backend역할_ChromaDB(VectorDB)



ChromaDB 기반 벡터 저장 및 유사도 검색 구현

사용자의 자소서나 포트폴리오에서 추출한 문장을 OpenAI Embedding으로 벡터화하고, 이를 ChromaDB에 저장하여 향후 유사도 기반 질의에 활용했습니다. 이로써 사용자가 입력한 질문이나 상황에 따라, 벡터 간 유사도를 계산해 가장 적합한 면접 질문 또는 회사/멘토를 실시간으로 추천하는 기능을 구현했습니다.

Backend역할(시스템 구성 및 로그인)

1. 프레임워크 및 서버 구성

FastAPI: RESTful API 서버 구성, 비동기 요청 처리, 라우터 및 응답 모델 설계.

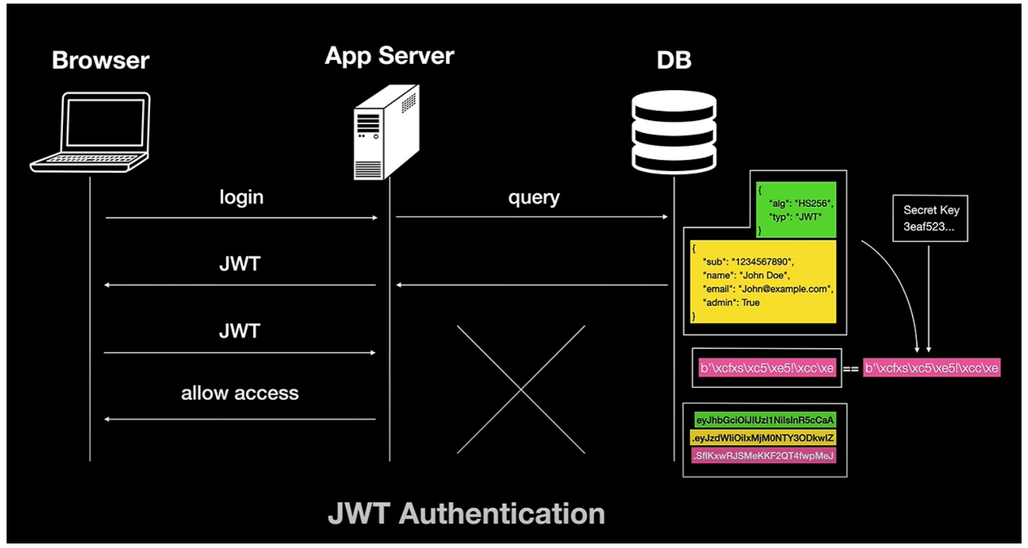
Uvicorn: ASGI 기반 FastAPI 앱 서버 실행.

multipart: 사용자 자소서/포트폴리오 PDF 업로드 처리용 미들웨어 지원 구현.

dotenv: 환경 변수(.env) 파일 로딩을 통한 API Key 및 DB 정보 관리.

CORS 설정: middleware.cors 모듈을 사용하여 React(클라이언트)의 특정 포트번호의 도메인만 허용.

DB 연동 : PostgreSQL 전용 async(비동기) 연결 / Vector화 사용하기 위해 정합성과 확장성을
-> Vector를 사용하기 위해서는 정합성과 학장성에 용이한 PostgreSQL을 선정하였습니다.



2. 로그인 및 보안

JWT를 Access Token과 Refresh Token 구조로 분리해 보안성을 강화 했습니다.

비밀번호는 bcrypt 해시를 사용해 암호화 저장.

Backend역할(사용자 자소서 첨부)

3. PDF 파일 처리

업로드 : UploadFile과 Form 방식을 사용한 자기소개서, 이력서, 포트폴리오 업로드 하였으며, 공백만 없애고 DB에 저장하게 구현했습니다.

텍스트 추출 : PyMuPDF를 사용해 레이어를 읽어 빠르고 정확한 문자열로 텍스트 추출하였습니다.

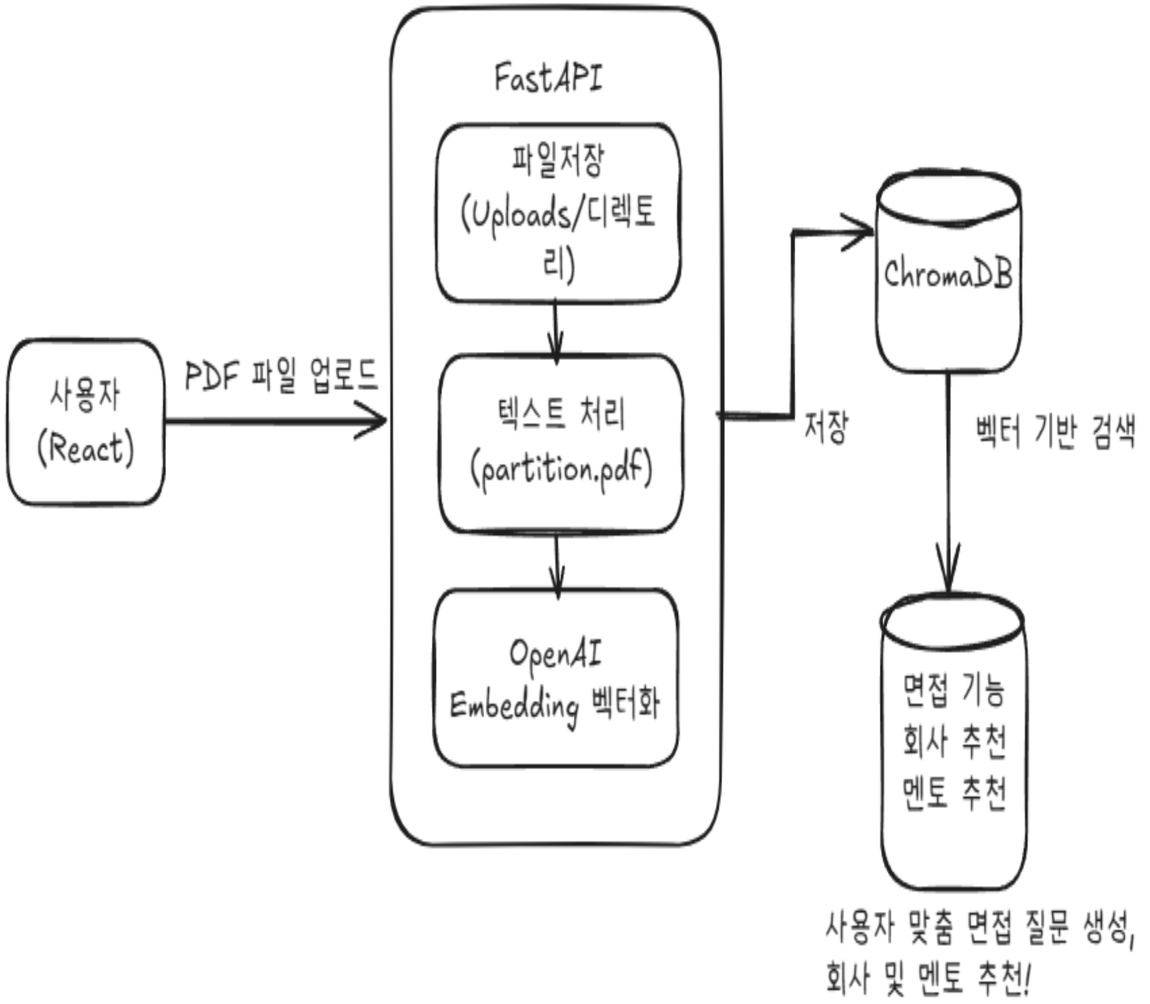
이미지 추출 :unstructured.partition.pdf라이브러리의 OCR 통해 한국어, 영어로 이미지 처리 했습니다.

4. PDF 파일 보안

파일저장 : 파일 업로드 시 서버의 디렉토리에 사용자 ID와 타임스탬프 기반으로 생성하여 저장함으로 파일 충돌을 방지하였고, DB에는 파일 경로, 텍스트 추출한 것만 저장하여 보안 강화.

MIME 타입 검사와 파일 크기 제한을 걸어두어 악성코드 방지에 강화했습니다.

예외처리 : 실패메시지



Backend역할(사용자 맞춤 면접 기능)

5. LLM & GPT 기반 AI 기능

5-1. OpenAI GPT API

사용자가 첨부한 자기소개서, 이력서, 포트폴리오에서 텍스트를 추출한 후, 이를 OpenAI Embedding 모델을 사용해 벡터로 변환합니다.
변환된 벡터는 ChromaDB 벡터스토어에 저장되며, 이후 면접 질문을 생성하거나 유사 항목을 검색할 때 벡터 간 코사인 유사도를 비교하여 맞춤형 면접질문이 생성되는 서비스입니다.

openai.ChatCompletion.create()
→ 키워드 기반 자기소개서 생성, 면접 질문/꼬리질문 자동 생성, 자소서 피드백.

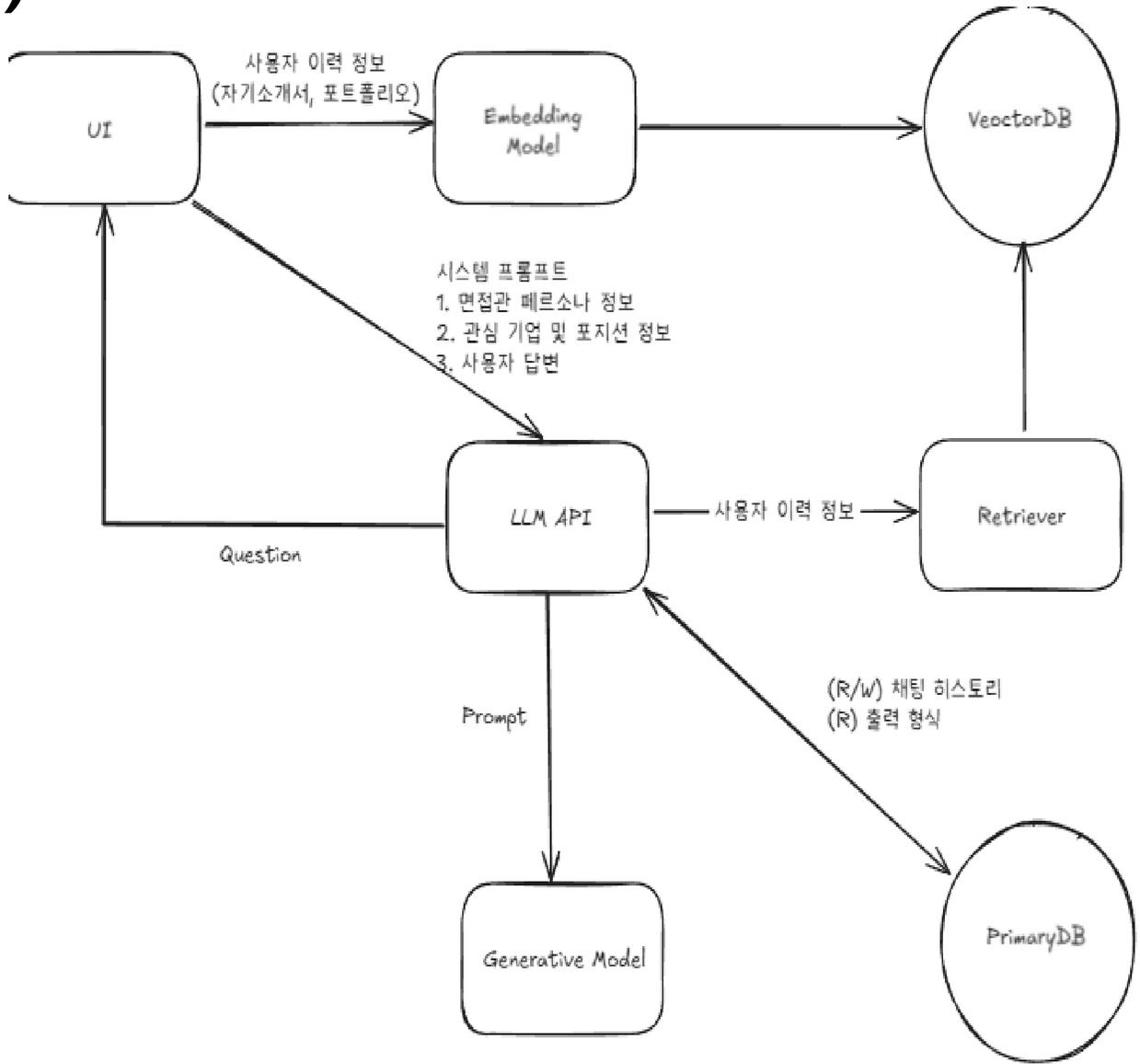
openai.Embedding.create()
→ 임베딩 후 코사인 유사도 검색 기반 추천 구현 (모델: text-embedding-ada-002)

5-2. LangChain

PromptTemplate: 질문 유형/면접관 스타일에 맞는 프롬프트 동적 생성

ConversationChain: AI 면접 대화 흐름 설계(질문 ↔ 응답 ↔ 꼬리질문)

Document, RetrievalQA, VectorStoreRetriever: 사용자 자소서를 벡터 DB와 연동하여 GPT가 유사 문장 검색 기반 질문 생성



Backend역할(회사 추천 및 멘토 추천)

6. LLM & GPT 기반 AI 기능

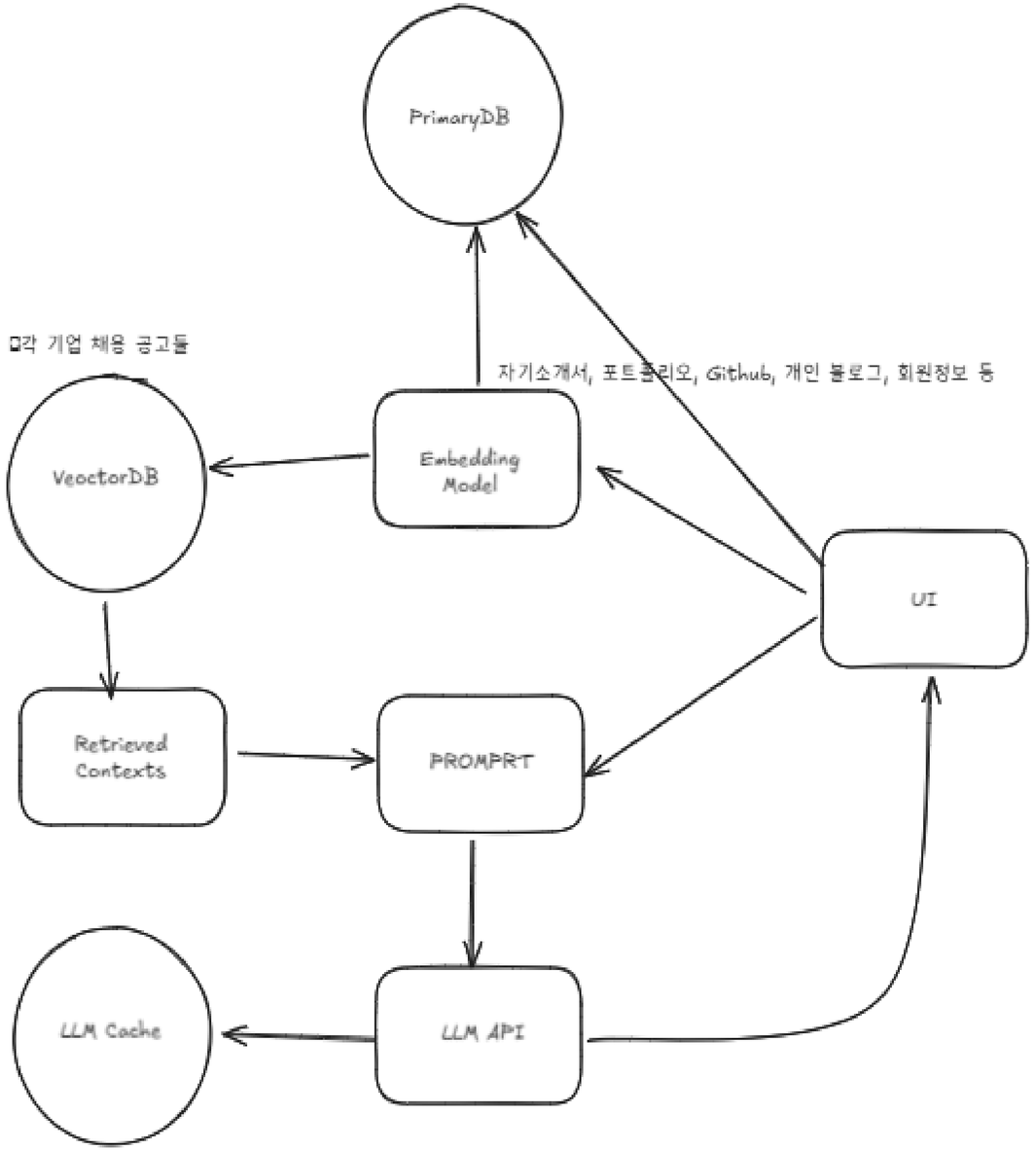
6-1. OpenAI GPT API

면접 기능과 동일한 방식으로 사용자가 입력 화면에 지역, 기술스택, 희망연봉 등 조건을 입력하면 코사인 유사도를 사용해 사용자에게 가장 어울리는 실시간 회사 및 멘토를 추천합니다.

openai.Embedding.create()
→ 첨부 파일과 입력 조건(지역, 기술스택, 연봉 등)을 결합한 후 벡터화 후 추천

6-2. LangChain

Document, RetrievalQA, VectorStoreRetriever: 사용자가 첨부한 자소서와 입력 조건을 기반으로 벡터화한 뒤, VectorStore에서 유사도 높은 기업 및 멘토 추천 결과 도출



Troubleshooting: 유사도 기반 벡터 추천 정확도 개선

임베딩 기반 유사도 추천 기능 구현 과정에서, 기대한 결과와 다른 공고가 추천되는 현상이 발생했습니다.
문제 원인은 사용자 입력값과 채용공고 설명의 벡터화 방식에 있었습니다.

초기에는 단순 문자열 조합만으로 임베딩했지만, 유사도 계산의 정확도가 떨어졌습니다.
이를 해결하기 위해 사용자 입력값을 "기술/지역/연봉" 형태로 구조화하고, 공고 설명도 제목 + 회사명 + 설명을 결합해 의미 밀도를 높였습니다.

또한 similarity <= 0 조건으로 무관한 결과를 제거하고, 유사도 기준 정렬 및 Top 10만 추출하여 추천 품질을 개선했습니다.

해당 코드는 OpenAI Embedding을 사용한 텍스트 벡터화와 코사인 유사도 계산을 기반으로 하며, 추천 시스템에서의 텍스트 전처리와 벡터 정렬 전략의 중요성을 실감한 경험입니다.

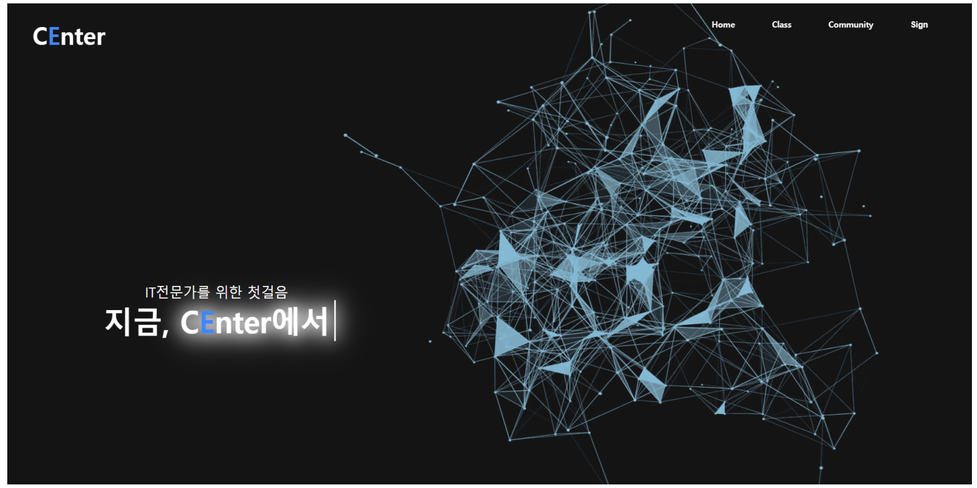
- get_embedding() → 텍스트 의미 보존을 위한 전처리 적용
- compute_cosine_similarity() → 직접 수식 계산으로 유사도 도출
- results.append() → 조건 필터링 + 정렬로 품질 향상
- .join(map(str, filter(None, [...])) 구문으로 의미 밀도 향상
- job description도 title + company + 내용으로 구성
- similarity <= 0 필터로 쓰레기 결과 제거
- sorted(..., reverse=True)[:10]로 상위 결과만 추출

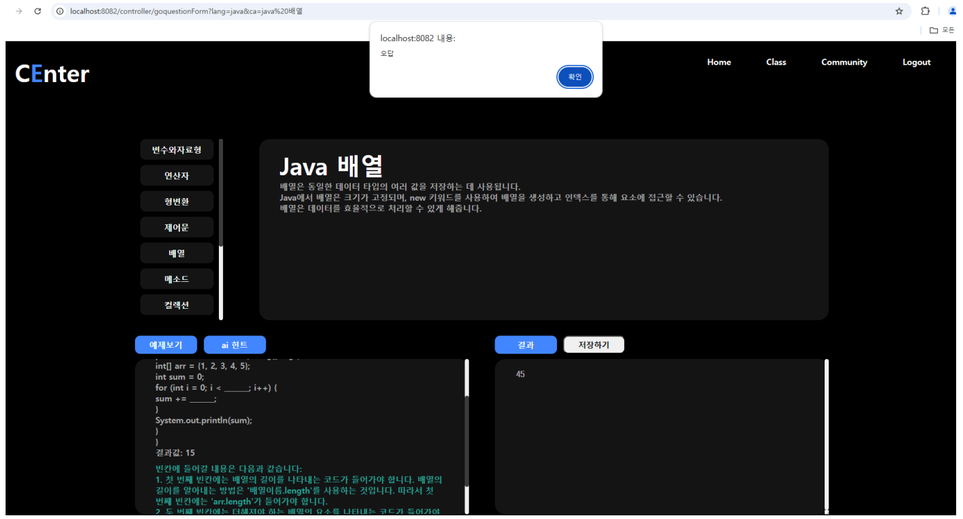
```python
def recommend_companies(tech_stack=None, location=None, salary=None):
    try:
        # ✅ 모든 입력을 문자열로 안전하게 변환
        combined_text = " ".join(map(str, filter(None, [tech_stack, location, salary]))).strip()
        user_vector = get_embedding(combined_text)
        if user_vector is None:
            raise ValueError("❌ 사용자 임베딩 생성 실패")

        tech_stacks = [stack.strip() for stack in tech_stack.split(",")] if tech_stack else []
        job_posts = crawl_saramin_jobs(tech_stacks, location, salary)
        if not job_posts:
            print("⚠️ 채용 공고가 없습니다.")
            return []

        results = []
        for idx, job in enumerate(job_posts):
            job_vector = get_embedding(job["description"])
            if job_vector is None:
                continue
            similarity = compute_cosine_similarity(user_vector, job_vector)
            if similarity <= 0:
                continue
            results.append({
                "id": idx,
                "name": job["company"],
                "techStack": tech_stacks if tech_stacks else ["정보 없음"],
                "location": location or "정보 없음",
                "link": job["link"],
                "similarity": round(float(similarity), 3)
            })

        return sorted(results, key=lambda x: x["similarity"], reverse=True)[:10]
```

```python
# 단일 텍스트 임베딩
def get_embedding(text: str) -> list:
    return embedding_model.embed_query(text)

# 코사인 유사도 계산
def compute_cosine_similarity(vec1: list, vec2: list) -> float:
    vec1 = np.array(vec1)
    vec2 = np.array(vec2)
    return np.dot(vec1, vec2) / (np.linalg.norm(vec1) * np.linalg.norm(vec2))
```

CEnter

주제	OPEN AI API를 활용한 AI 코딩 학습 도우미
개발기간	2025.02.18 ~ 2025.03.05
개발인원	4명
개발환경	VS Code, Spring, MySQL
서버/API	Apache Tomcat & OPEN AI API
사용언어	Java, Html, CSS, Javascript, Python





주요기능

01. 사용자가 선택한 언어에 맞춰 AI가 적절한 코딩 문제를 자동으로 생성

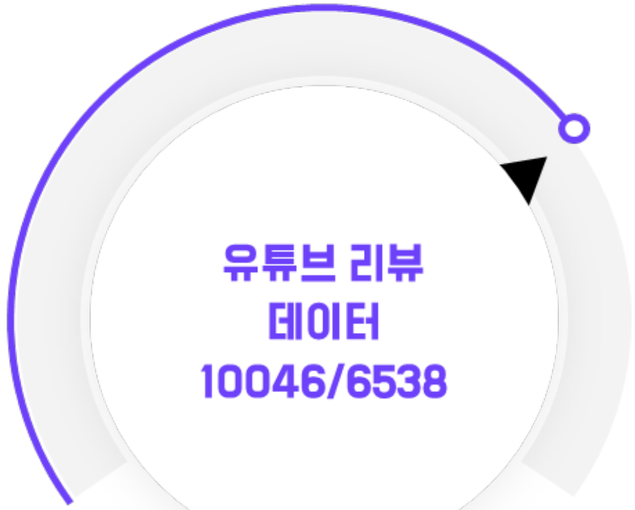
02. AI가 문제의 정답을 판단하고, 사용자가 모를 경우 힌트 제공

03. 사용자의 입력 값이 정답인지 자동으로 채점

04. 문제를 저장하여 사용자가 다시 복습할 수 있도록 지원

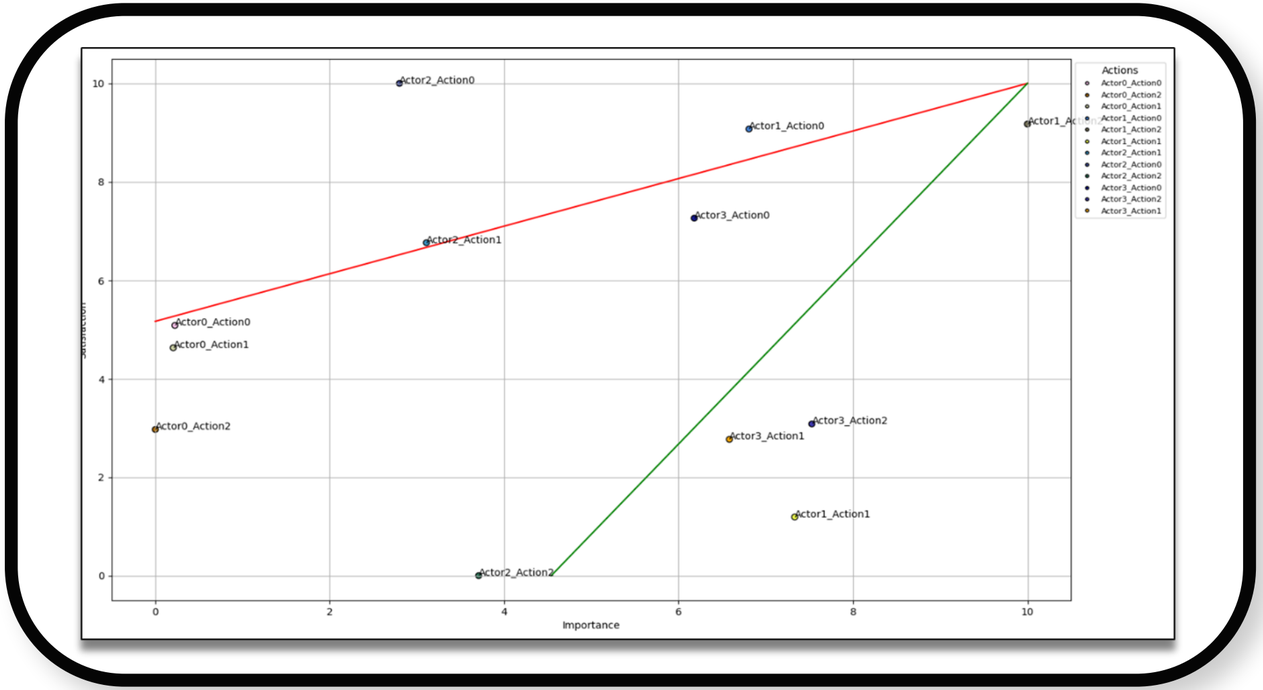
05. 게시판을 통해 다양한 코딩 문제를 사용자들끼리 공유하고 소통

GitHub https://github.com/2024-SMHRD-DCX-BigData-16/CEnter.git



total: 10046

process: 6538



"코딩" 키워드 크롤링을 통한 전처리 후 TF-IDF 기반 벡터화 및 KMeans 클러스터링, RandomForest를 통한 특성 중요도 분석을 수행하였고, 중요도는 높지만 상대적으로 언급이 적은 '입문자 도움 필요' 클러스터를 기반으로 코딩 도우미 서비스 기획 하게 되었습니다.

cluster0 - 학습 방법 공유

학습을 하기 위해 퀴즈, 영상 게시판 등을 이용해요!!

cluster1 - 도움 필요(입문)

기초 학습을 어떻게 하면 좋을까요, 최신 기술 및 심화 학습이 필요해요

cluster2 - 도움 필요(학생)

학습 내용 요약본 활용 및 효율적인 알고리즘 팁을 알고 싶어요

cluster3 - 실무 경험 필요

실전 위주의 학습 방법 및 학습 플랫폼을 활용하여 트렌드 있는 지식을 소통하고 싶어요

핵심프로젝트 CEnter

로그인 및 회원가입 + 게시판

HTML, CSS, JavaScript를 이용해 로그인 및 회원가입 UI 구축 + 게시판(커뮤니티) 서비스 기능 구현

Request

Response

Server(Tomcat)

회원가입 및 로그인 입력한 데이터는 Spring(MVC) 컨트롤러에 전송 + Tomcat 환경에서 서버 설정 및 세션(Session) 기반 로그인 처리 및 인증 관리 구현

Mybatis

테이블 명세서 바탕으로 테이블, 컬럼 구현

DB구축

MyBatis를 사용하여 DB 연동 (SQL 매퍼 XML 구현)

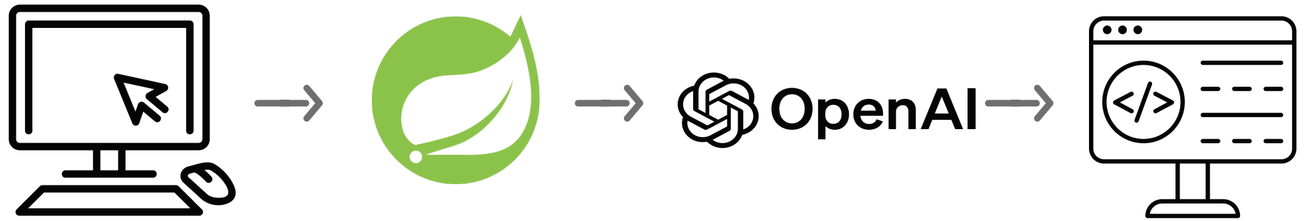
핵심프로젝트 CEnter

OpenAI

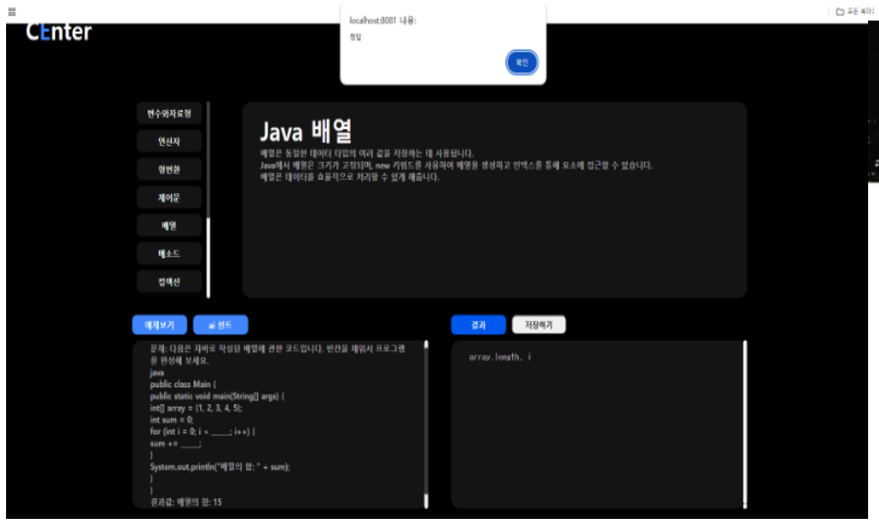
OpenAI API 연동 문제 생성

Spring MVC 기반으로 사용자가 프로그래밍언어와 원하는 주제를 선택하면 백엔드에서 OpenAI API를 호출하여 코딩 문제를 생성합니다. 생성된 문제는 JSP를 통해 사용자에게 제공되며, 학습 도구로 활용할 수 있습니다. 또한 문제에 대한 힌트를 함께 제공하고, 사용자가 푼 문제는 개인 페이지에 선택적으로 저장하여 복습할 수 있도록 설계하였습니다.

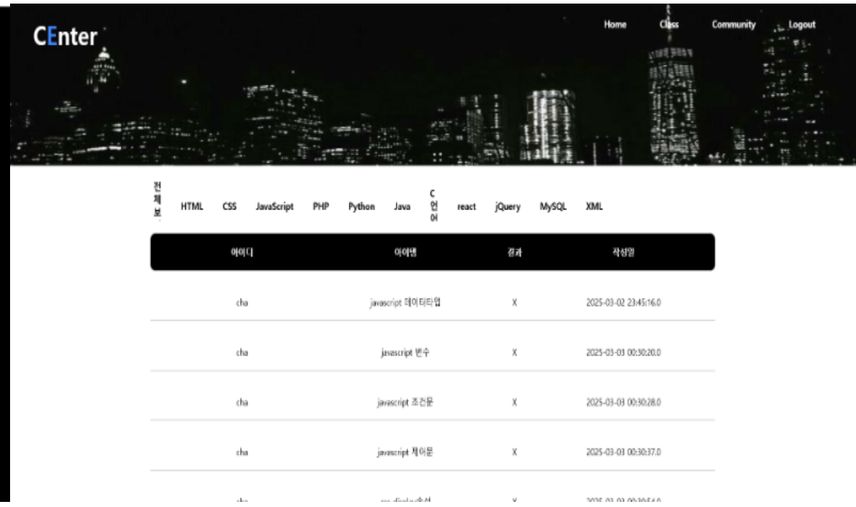
입문자 맞춤형 코딩 학습도우미



문제 생성 → 백엔드 처리 → API 호출 → 결과 출력



문제 출력 화면



저장된 문제 화면

핵심프로젝트 CEnter

Troubleshooting: 불완전한 GPT 응답 처리 로직 개선

GPT 응답의 구조가 고정되어 있지 않아, 예외 처리를 어떻게 설계할지 고민이 많았던 부분입니다. 초기에는 단순히 response.choices[0].text만 추출했지만, 종종 불완전한 응답이나 코드가 아닌 문장이 혼합된 응답이 반환되어, 조건 분기 및 텍스트 정제 로직을 직접 추가하면서 안정성을 확보했습니다.

문제 코드 : response.choices[0].text;

개선 코드 : 정규식을 통한 코드 블럭 추출

- 예외 상황까지 커버하는 구조 (?., 조건 분기, fallback)

- .split → .map → .filter → .join으로 포맷 정제

- console.log() 로 실제 디버깅용 출력까지 추가

```
// ✅ GPT 응답에서 코드 블럭만 안전하게 추출하는 로직
const fullText = response.choices?.[0]?.text || "";

// 1. ```로 감싼 코드 블럭을 추출 (언어명 포함 여부 무시)
const codeRegex = /```(?:[a-zA-Z]*\n)?([\s\S]*?)```/;
const codeMatch = fullText.match(codeRegex);

// 2. 코드 블럭 추출 성공 시, 불필요한 줄바꿈/공백 정리
let extractedCode;
if (codeMatch && codeMatch[1]) {
extractedCode = codeMatch[1]
    .split("\n")
    .map(line => line.trimEnd())
    .filter(line => line !== "")
    .join("\n");
} else {
  // 3. 코드 블럭이 없을 경우: 전체 텍스트에서 fallback 처리
  extractedCode = fullText
    .replace(/(?:^|\n)(?!.*;).*/g, "") // 설명 문장 제거 (세미콜론 없는 줄)
    .trim();
}

// ✅ 최종적으로 문제 코드로 사용할 텍스트
console.log("✨ 파싱된 문제 코드:", extractedCode);
```

읽어주셔서 감사합니다 :)

PARK MyungHoon

MYUNG HOON PARK

010 - 4183 - 1640

audgns1640@gmail.com